第5章 8086汇编语言程序设计

西南交通大学计算机与人工智能学院

keli@home.swjtu.edu.cn

本章主要内容

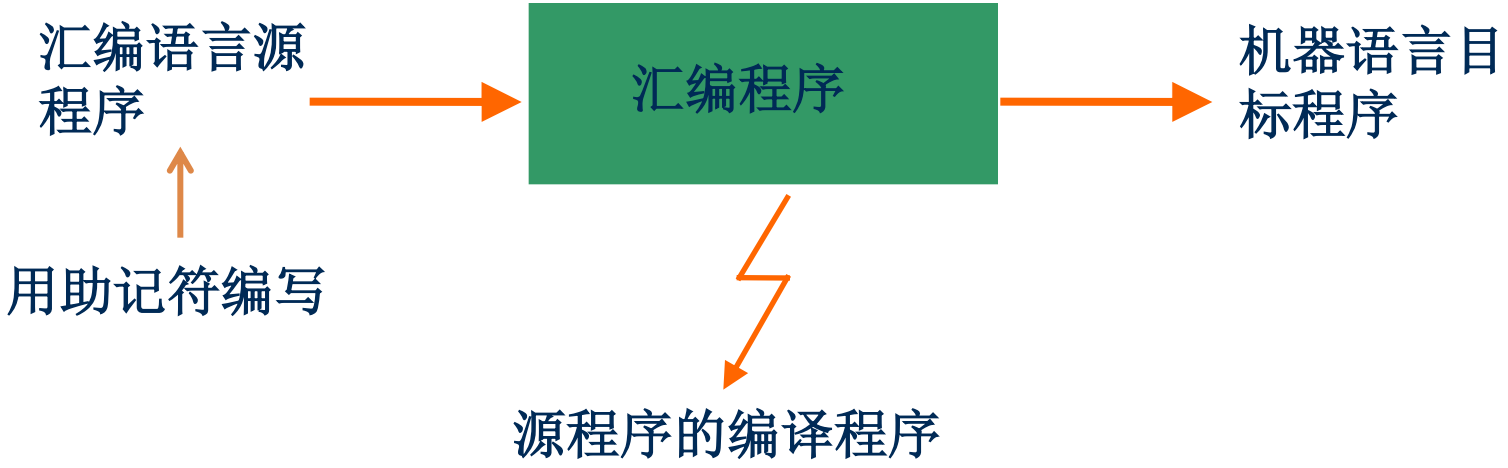
1. 汇编语言基础知识
2. 汇编语言的伪指令
3. 系统功能调用
4. 汇编语言程序设计
5. 汇编语言程序的上机过程

5.1.1汇编语言

机器语言 → 汇编语言 → 高级语言

- 汇编语言（**Assembly Language**）是一种用符号表示的面向机器的程序设计语言,不同**CPU**的计算机有不同的汇编语言。
- 比机器语言易于阅读、编写和修改，能充分利用计算机的硬件资源，占用内存空间少。
- 常用于计算机控制系统的开发和高级语言编译程序的编制等应用场合。

汇编程序



汇编语言程序设计与执行过程

D:\masm>EDIT → 输入汇编语言源程序 → 源文件 .ASM

D:\masm>MASM → 汇编（编译） → 目标文件 .OBJ

D:\masm>LINK → 链接 → 可执行件 .EXE

D:\masm>TD → 调试 → 最终程序

源程序行号

相对于段地址的偏移地址

存储单元内容（目标码）

源程序

```
1   0000                    DATA   SEGMENT
2   0000    66                X     DB     66H
3   0001    35                Y     DB     35H
4   0002    ????              Z     DW     ?
5   0004                    DATA   ENDS
6   0000                    CODE   SEGMENT
7                         ASSUME   CS:CODE,DS:DATA
8   0000     B8 ---- R       MOV   AX,DATA
9   0003     8E D8           MOV   DS,AX
10  0005     33 C0           XOR   AX,AX
11  0007     A0 0000 R       MOV   AL,X
```

内存中段地址、偏移地址与单元内容的对应关系

DS：	0000H	66H
	0001H	35H
	0002H	? ?
	0003H	? ?
		∘ ∘
	000FH	
CS：	0000H	B8H
	0001H	??H
	0002H	??H
	0003H	8EH
	0004H	D8H
	0005H	33H
	0006H	C0H

```
12   000A   D1 E0          SHL   AX,1
13   000C   8B D8          MOV   BX,AX
14   000E   D1 E0          SHL   AX,1
15   0010   D1 E0          SHL   AX,1
16   0012   03 D8          ADD   BX,AX
17   0014   B4 00          MOV   AH,0
18   0016   A0 0001 R      MOV   AL,Y
19   0019   D1 E0          SHL   AX,1
20   001B   D1 E0          SHL   AX,1
21   001D   03 D8          ADD   BX,AX
22   001F   D1 EB          SHR   BX,1
23   0021   89 1E 0002 R   MOV   Z,BX
24   0025   B4 4C          MOV   AH,4CH
25   0027   CD 21          INT   21H
26   0029                CODE    ENDS
27                         END
```

000CH	8BH
000DH	D8H
000EH	D1H
000FH	E0H
0010H	D1H
0011H	E0H
0012H	03H
0013H	D8H
0014H	B4H
0015H	00H
0016H	A0H
0017H	01H
0018H	00H
0027H	CDH
0028H	21H

```
Microsoft (R) Macro Assembler Version 5.00      Symbols-1
Segments and Groups:
N a m e                 Length      Align       Combine    Class
CODE . . . . . . . . .  0029        PARA        NONE
DATA . . . . . . . . .  0004        PARA        NONE
Symbols:
N a m e                 Type        Value       Attr
X   . . . . . . .. . . .  L BYTE     0000        DATA
Y   . . . . . . . . .. .  L BYTE     0001        DATA
Z   . . . . . . . . .. .  L WORD     0002        DATA
@FILENAME   . . . . . . .   TEXT    EXAMP1
        27 Source   Lines
        27 Total    Lines
        7 Symbols
   50388 + 354044 Bytes symbol space free
         0 Warning Errors
         0 Severe  Errors
```

5.1.2 汇编语言源程序结构

```
数据段名    SEGMENT
            …
数据段名    ENDS

附加段名    SEGMENT
            …
附加段名    ENDS
```

```
堆栈段名  SEGMENT
            …
堆栈段名  ENDS

代码段名  SEGMENT
            …
代码段名  ENDS
```

- 例　编写汇编程序，计算2010H +2011H，并把和存入RESULT单元。

```
DATA   SEGMENT              ;定义数据段
     X   DW   2010H         ;定义被加数
     Y   DW   2011H         ;定义加数
     RESULT   DW    ?       ;分配和数存放单元
DATA   ENDS                 ;数据段定义结束

CODE   SEGMENT
               MAIN   PROC    FAR
ASSUME         CS:CODE，DS:DATA
     START:    PUSH   DS
               MOV    AX,0
               PUSH   AX
               MOV    AX,DATA
               MOV    DS,AX
```

```
            MOV   AX,X
            ADD   AX,Y
            MOV   RESULT,AX
            RET
    MAIN   ENDP
CODE   ENDS
    END   START
```

***.EXE 文件与*.COM 文件的区别：**

*.COM 文件的代码区、数据区、堆栈区同处于一个段内（64K 空间内），即 CS=DS=ES=SS；另外 IP 初值=0100H

*.EXE 文件的代码区、数据区、堆栈区则分处于不同的段内。即 DS=ES≠CS≠SS；另外 IP 初值=0000H

5.1.3 汇编语言语句

- 指令性语句 → CPU执行的语句，能够生成目标代码
- 指示性语句 → CPU不执行，而由汇编程序执行的语句，不生成目标代码

  伪指令

指令性语句格式：

［标号：］ ［前缀］ 助记符［操作数］，［操作数］［ ；注释］

指令的符号地址，
标号后要有冒号

操作码

注释前加分号

指示性语句格式：

［名字］　伪指令助记符　操作数［，操作数，…］［ ；注释］

变量的符号地址，其后不加冒号

指示性语句中至少有一个操作数

标号、名字

- 标号后有冒号，在指令性语句前；名字后不加冒号，在指示性语句前。
- 英文字母、数字及专用字符组成,最大长度不能超过31个英文字符,且不能由数字打头,不能用保留字(如寄存器名,指令助记符,伪指令)。

操作码

指令码 → 命令CPU要完成的操作

伪指令助记符
- 指示汇编程序要完成的操作
- 用来定义变量，分配存储单元
- 指示程序开始和结束等

注释

- 用分号作为注释的开始
- 用于说明程序或语句的功能，以便于阅读和理解

操作数

- 寄存器
- 存储器单元
- 常量
- 变量或标号
- 表达式

5.1.4汇编语言的数据

- 常量
  - 数字常量
  - 字符串常量：用引号引起的字符或字符串
  - 例：‘A’，’ABCD’

↓

汇编时被译成对应的ASCII码41H，42H，43H，44H

- 变量
  - 存储单元的符号地址，其存储单元的内容在程序运行时可以被修改。
  - 变量的属性：

    段属性 ——变量所在段的段地址

    偏移属性 ——变量所在段的段内偏移量

    类型属性 ——变量所占单元的字节数（字节、字和双字）

- 标号
  - 指令的符号地址，可用作控制转移指令的操作数。
  - 标号的属性：

    段属性 ——标号所在段的段地址

    偏移属性 ——标号所在段的段内偏移量

    类型属性 ——距离属性，表示标号可以为段内或段间

5.1.5 操作符与表达式

- 算术运算
- 逻辑运算
- 关系运算
- 取值运算和属性运算
- 其它运算

1.算术运算和逻辑运算符

- 算术运算符 +，-，*，/，MOD
- 逻辑运算符 AND，OR，NOT，XOR
- 例：　MOV　AL，8 AND 4

  MOV　AL，8+4-1

2.关系运算符

EQ 相等
NE 不相等
LT 小于
GT 大于
LE 小于等于
GE 大于等于

参与关系运算的必须是两个数值：

关系不成立，结果为0；

关系成立，结果为FFFFH。

3.数值回送运算符（取值运算符）

用于解析标号/变量的数值或属性

- **OFFSET** 取得其后变量或标号的偏移地址
- **SEG** 取得其后变量或标号的段地址

- **TYPE** 取变量的类型
- **LENGTH** 取所定义存储区的长度
- **SIZE** 取所定义存储区的字节数

数值回送运算符例

```
MOV   AX，SEG   DATA
MOV   DS，AX
MOV   BX，OFFSET   DATA
```

等价于 → `LEA   BX，DATA`

数值回送运算符例

若BUFFER存储区是用如下伪指令定义：

BUFFER　DW　200 DUP(0)

则：

TYPE　　BUFFER　等于2

LENGTH　BUFFER　等于200

SIZE　　BUFFER　等于400

4.属性运算符

PTR THIS SHORT HIGH LOW

属性运算符**PTR**，用于指定其后存储器操作数的类型

例：

MOV BYTR PTR[BX]，12H

5.其它运算符

- 方括号：

  [　　] —— 方括号中内容为操作数的偏移地址

- 段重设符

  段寄存器名：[　　]—— 用于修改默认的段基地址

5.2 伪指令

伪指令作用

- 用于定义变量、分配存储区、定义逻辑段、指示程序开始和结束等

常用伪指令

- 数据定义伪指令
- 符号定义伪指令
- 段定义伪指令
- 过程定义伪指令
- 结束伪指令

5.2.1 变量定义伪指令

1. 用于定义数据区中变量的类型

2. 格式：

变量名　伪指令助记符　操作数，…　　；注释

某些情况下可省略

可选

3. 伪指令助记符

- DB　　定义的变量为字节型
- DW　　定义的变量为字类型（双字节）
- DD　　定义的变量为双字型（4字节）
- DQ　　定义的变量为4字型（8字节）
- DT　　定义的变量为10字节型

例：

```
DATA1    DB   11H，22H，33H，44H
DATA2    DW   11H，22H，3344H
DATA3    DD   11H*2，22H，33445566H
```

以上变量在内存中的存放形式

- 伪指令的性质决定所定义变量的属性
- 定义字符串必须用DB伪指令

例：

DATA1　DB　‘ABCD’，66H

4.重复操作符

- 为一个数据区的各单元设置相同的初值
- 格式：

[变量名]　伪指令助记符　n　DUP（初值，...）

例：

```
DIS    DB   20   DUP（0）
       DB   3   DUP（22H，11H，？）
```

预留一个字节的内存单元（随机数）

5.用“？”预留存储空间

```
MEM1        DB   34H，'A'，？，？，？
            DW   20   DUP（？）
```

↓

预留40个字节单元

5.2.2 符号定义伪指令

- 符号包括汇编语言的变量名、标号名、过程名等。
- 常用符号定义伪指令有：

EQU

“=”

LABEL

1.EQU伪指令

- 格式：名字　EQU　表达式
- 功能：给表达式赋予一个名字。定义后，可用名字代替表达式。在同一源程序中，一个名字只能用EQU定义一次。
- 例如：

```
PIX   EQU   64*1024     ;名字PIX代表数值表达式的值
A     EQU   7
B     EQU   A-2
```

2.等号（=）伪指令

- 格式：名字 = 表达式
- 功能：与EQU基本相同，区别是它可以对同一个名字重新定义。
- 例如：

```
COUNT = 10
MOV   AL,COUNT
…
COUNT = 5
…
```

3. LABEL伪指令

- 格式：变量/标号　LABEL　类型
  - 变量的类型有：BYTE、WORD、DWORD、DQ、DT
  - 标号的类型有：NEAR、FAR。
- 功能：定义变量或标号的属性类型，而变量或标号的段属性和偏移属性由该语句所处的位置确定。

- 例如：利用LABEL使同一个数据区有一个以上的类型及相关属性。

```
AREAW   LABEL   WORD        ；AREAW与AREAB指向相同的数据区,
                            ；AREAW类型为字,AREAB类型为字节
AREAB   DB   100 DUP(? )
          …
MOV   AX,2011H
MOV   AREAW,AX        ；(AREAW)=2011H
          …
MOV   BL,AREAB        ；BL=11H
```

5.2.3 段定义伪指令

1. 段定义伪指令

- 说明逻辑段的起始和结束
- 说明不同程序模块中同类逻辑段之间的联系形态

段名　SEGMENT［定位类型］［组合类型］［‘类别’］

段名相同

⋮

段名　ENDS

说明逻辑段的边界

说明不同模块中同名段的组和连接方式

在连接时决定各逻辑段的装入顺序

[ ]括号中的内容为可选项，告诉汇编程序和连接程序如何确定段的边界、如何连接几个程序模块

(1)定位类型:

说明段的起始地址应有怎样的边界值

- BYTE: 段的起点从存储器任何地址开始,这种类型段间不留空隙，存储器利用率高。
- WORD: 段的起点从偶地址开始
- PARA: 定位类型的默认值为PARA。段的起点从节边界开始（16个字节为1节）,所以起始地址必为16的倍数,段间往往有空隙。
- PAGE: 段的起点从页边界开始（256个字节为1页），其起始地址必为256的倍数。

(2)组合类型

- PUBLIC：本段与其他模块中说明为PUBLIC的同名同类别的段链接起来，公用一个段地址，形成一个新的逻辑段，所有偏移量调整为相对于新逻辑段起始地址的值。
- STACK：本段与其他模块中说明为STACK的同名的堆栈段链接起来，公用一个段地址，形成一个新的逻辑段。同时，系统自动初始化SS及SP。
- COMMON：同名段从同一个内存地址开始装入。所以，各个逻辑段将发生覆盖。连接以后，该段长度取决于同名段中最长的那个，而内容有效的是最后装入的那个。

(2)组合类型

- MEMORY：与PUBLIC同义，只不过MEMORY定义的段装在所有同名段的最后。若连接时出现多个MEMORY，则最先遇到的段按组合类型MEMORY处理，其他段组合类型按PUBLIC处理。
- PRIVATE：不组合，该段与其他段逻辑上不发生关系，即使同名，各段拥有各自的段基值。组合类型的默认值为PRIVATE。
- AT exp：段地址为表达式exp的值（长度为16位）。此项不能用于代码段。例如：AT 0530H，表示本段从物理地址0530H开始。

(3)类别

- 当几个程序模块进行连接时，其中具有相同类别名的段按出现的先后顺序被装入连续的内存区。
- 没有类别名的段，与其他无类别名的段一起连续装入内存。
- 典型的类型名有：“STACK”，“CODE”，“DATA”。

段定义伪指令例

```
DATA   SEGMENT    PUBLIC   'CODE'
MEM1   DB   11H，22H
DATA   ENDS
```

2. 设定段寄存器伪指令

- 说明所定义逻辑段与段寄存器的关系。
- 格式

ASSUME　段寄存器名：段名[，段寄存器名：段名，...]

  - 段寄存器可以是：CS、DS、ES、SS。
  - ASSUME伪指令只是指示各逻辑段使用段寄存器的情况，并没有对段寄存器的内容进行赋值。
  - DS、ES的值必须在程序段中用指令语句进行赋值，而CS、SS由系统负责设置，程序中也可对SS进行赋值，但不允许对CS赋值。

5.2.4 过程定义伪指令

- 用于定义一个过程体
- 格式：

```
过程名    PROC   [ NEAR / FAR ]
                  ⋮
                  RET
过程名    ENDP
```

- 过程名按汇编语言命名规则设定，汇编及链接后，该名称表示过程程序的入口地址，供调用使用。

5.2.5 模块定义结束伪指令

- **NAME**——为目标程序设定一个名字
- 格式：

  NAME 模块名

- **TITLE**——为程序清单指定打印标题
- 格式：

  TITLE 标题名 （**TITLE**伪指令可指定每一页上打印的标题。标题最多可用**60**个字符。）

- END——表示源程序结束
- 格式：

  END ［标号］

一个完整源程序结构例

```
DSEG    SEGMENT
DATA1   DB  1，2，
DATA2   DW  1234H
DSEG    ENDS

ESEG    SEGMENT
        DB  20  DUP（？）
ESEG    ENDS

SSEG    SEGMENT  STACK  ‘STACK’
        DB  200  DUP（？）
SSEG    ENDS
```

```
CSEG SEGMENT
        ASSUME  CS：CSEG，DS：DSEG，
                ES：ESEG，SS：SSEG
START：MOV  AX，DSEG
        MOV  DS，AX
        MOV  AX，ESEG
        MOV  ES，AX
        MOV  AX，SSEG
        MOV  SS，AX
         ⋮
CSEG  ENDS
        END  START
```

5.2.6 其他伪指令

- ORG——指定程序或数据区在置入内存时相对于 各段地址的起始偏移量。
- 格式：

  ORG　表达式

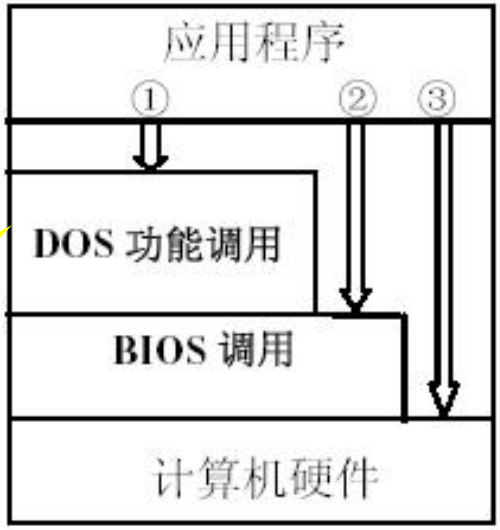
例：

ORG　2000H

5.3 功能调用

- DOS 功能调用 → 高级调用，操作系统提供
- BIOS功能调用 → 低级调用

应用程序与DOS及BIOS调用关系图



编程时遵循的原则：

①尽可能使用DOS的系统功能调用，以降低编程的复杂性。

②在DOS功能不能实现情况下考虑用BIOS功能调用。

③在DOS和BIOS的中断子程不能解决问题时使用IN/OUT指令直接控制硬件。

5.3.1 DOS 功能调用

- 设备管理
- 目录管理
- 文件管理
- 其 他

- 包含多个子功能的功能包，各子功能采用功能号来区分；
- 功能调用格式：

[置入口参数]

```
MOV   AH，功能号        ；功能调用号送AH寄存器
INT   21H               ；执行软终端指令
```

1.从键盘输入单字符并显示

- 调用方法：

```
MOV  AH，01
INT  21H
```

- 功能：接收从键盘输入的一个字符并在屏幕回显。输入字符的ASCII码存在AL中

1.单字符输入

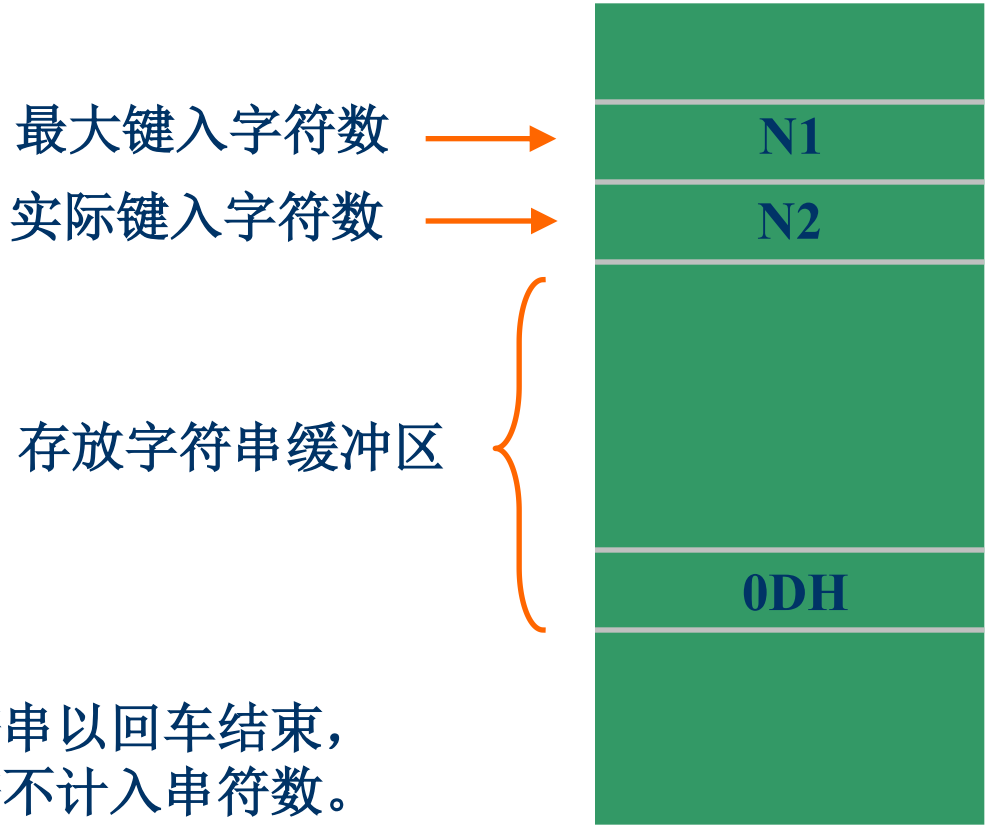
```
GET_KEY:  MOV  AH,1
          INT  21H
          CMP  AL,'Y'
          JZ   YES
          CMP  AL,'N'
          JZ   NO
          JNZ  GET_KEY
     YES:   ┆
      NO:   ┆
```

交互式应答程序

2. 从键盘输入字符串（书P90）

- AH=0AH ← 功能调用号0AH
- DS：DX ← 字符串在内存中的存放地址
- INT　21H

- 用户自定义缓冲区格式：



- 输入字符串以回车结束，且回车符不计入串符数。

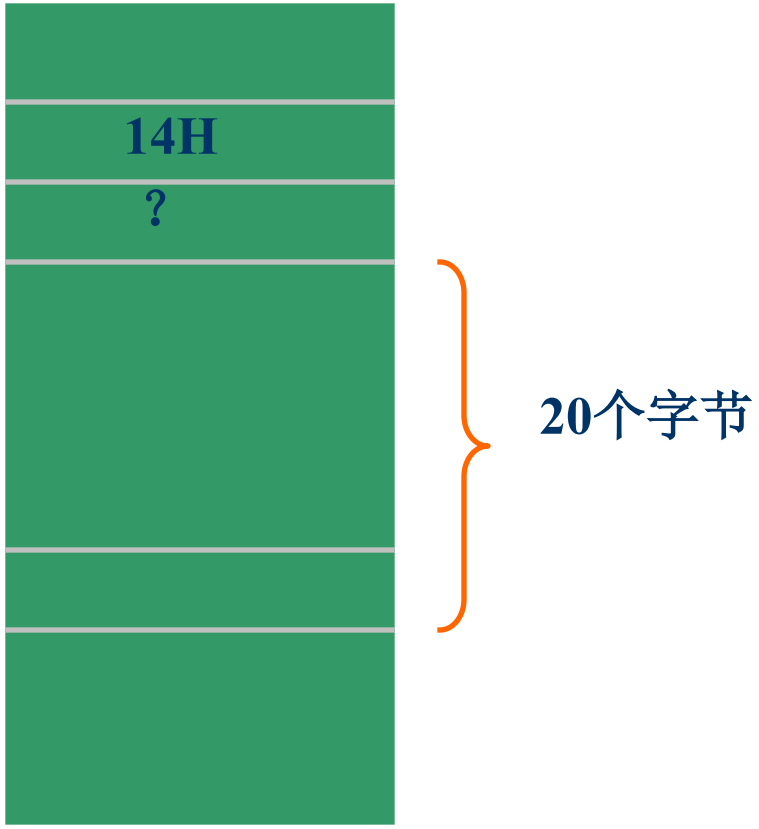
输入字符串程序段

```
DAT1   DB  20，？，20  DUP（？）

LEA  DX，DAT1
MOV  AH，0AH
INT  21H
```

在数据段中定义

定义后的输入缓冲区初始状态：



3. 单字符显示输出

```
MOV DL，41H        ；待输出字符ASCII码→DL
MOV AH，02H        ；功能号02H→AH
INT 21H
```

执行结果：
屏幕显示A

4. 字符串输出显示

```
DATA     SEGMENT
MESS1   DB  'Input String:' 0DH, 0AH,'$'
                                ; 被显示的字符串必须以'$'结束
DATA   ENDS
CODE   SEGMENT
          ⋮
MOV     DX, SEG  MESS1 ; 待输出字符串首址的段基址→ DS
MOV     DS, DX
MOV     DX, OFFSET  MESS1 ; 待输出字符串首址的偏移地址→ DX
MOV     AH, 09          ; 功能号09H→AH
INT     21H
          ⋮
```

5. 进程终止

功能：结束当前程序，返回DOS。

```
MOV  AH,4CH          ;功能调用号4CH→AH。
INT  21H
```

5.4 汇编语言程序设计

设计步骤：

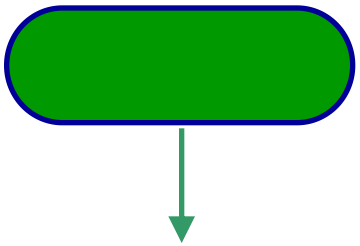
- 根据实际问题抽象出数学模型
- 确定算法
- 画程序流程图
- 分配内存工作单元和寄存器
- 程序编码
- 调试
- 文档整理

汇编语言程序有3种基本结构：

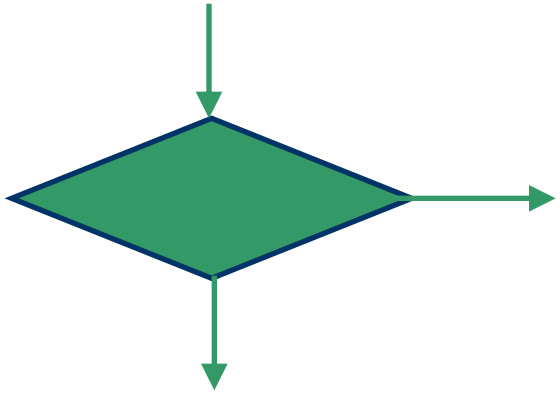
- 顺序结构
- 分支结构
- 循环结构

程序流程图符号

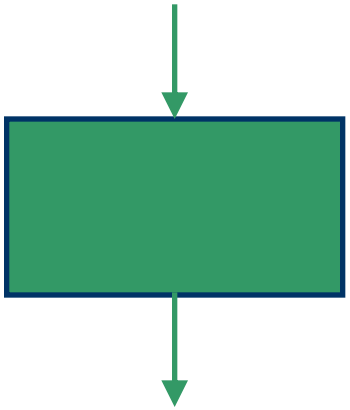
起始：



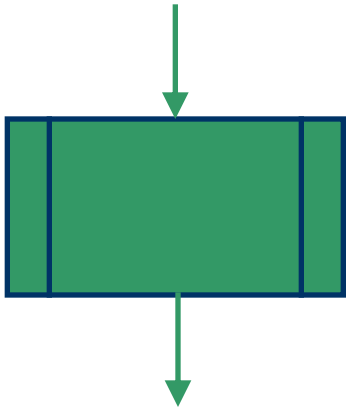
判断：



功能：



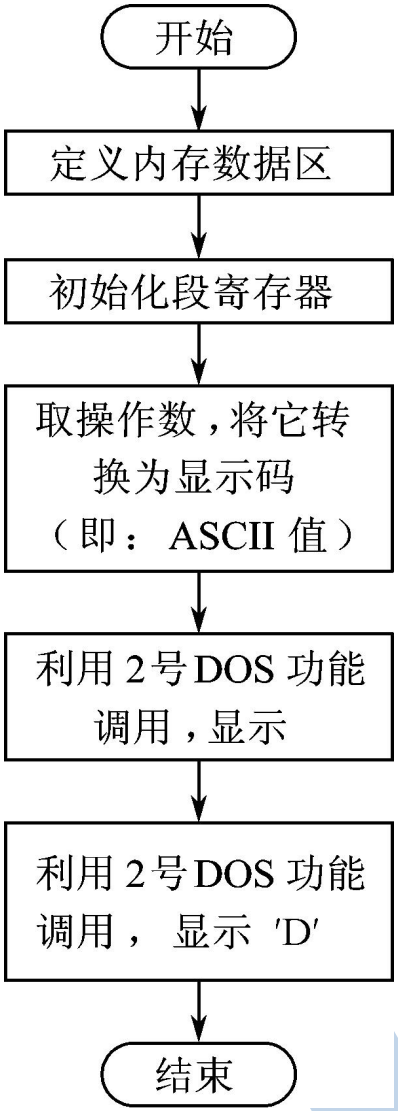
子过程：



顺序结构程序设计举例

- 【例5-2】编程将内存数据段字节单元INDAT存放的一个数n（假设0≤ n ≤9 ），以十进制形式在屏幕上显示出来。

```
DATA   SEGMENT                    ;数据段定义
        INDAT  DB 8
DATA   ENDS
CODE  SEGMENT                     ;代码段定义
  ASSUME CS:CODE,DS:DATA
START:
      MOV        AX,DATA
      MOV        DS,AX            ;初始化DS
```



```
        MOV     DL,INDAT
        OR      DL,30H
        MOV     AH,2
        INT     21H
        MOV     DL,'D'
        MOV     AH,2
        INT     21H
        MOV     AH,4CH
        INT     21H
CODE    ENDS
 END    START
```

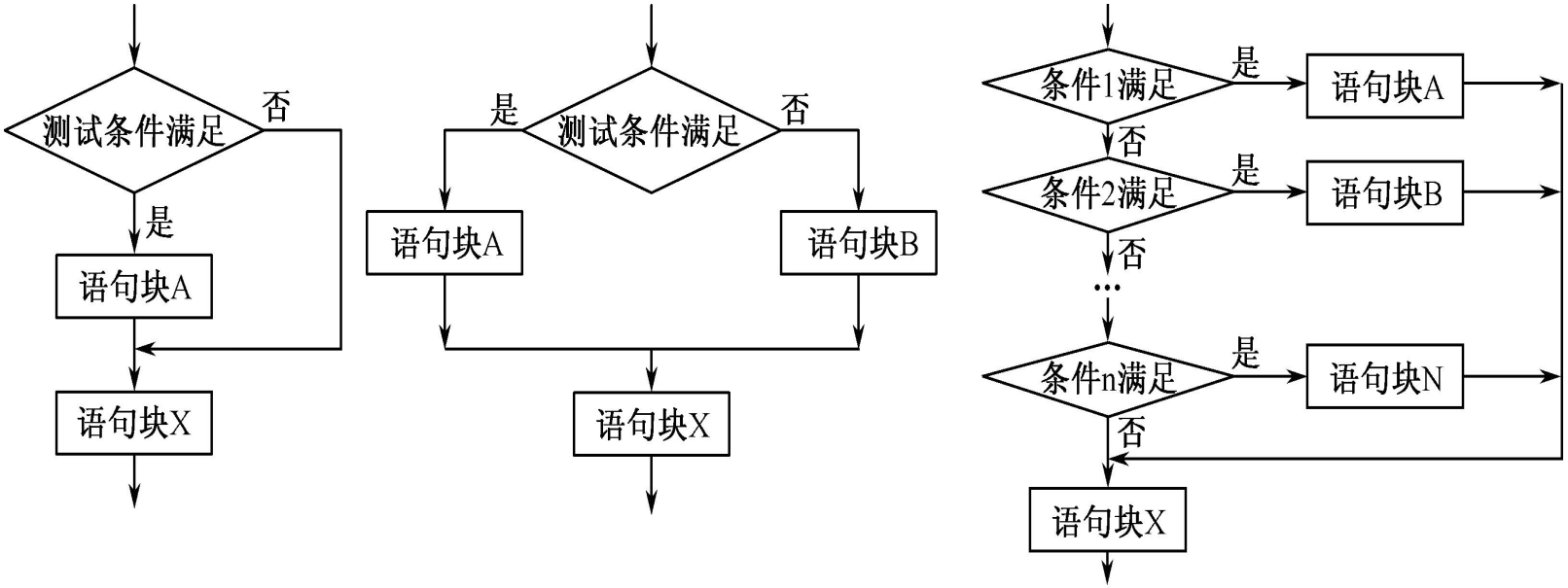
要显示的字符的ASCII码存于DL

DOS功能调用：单字符输出

DOS功能调用：单字符输出

DOS功能调用：进程终止

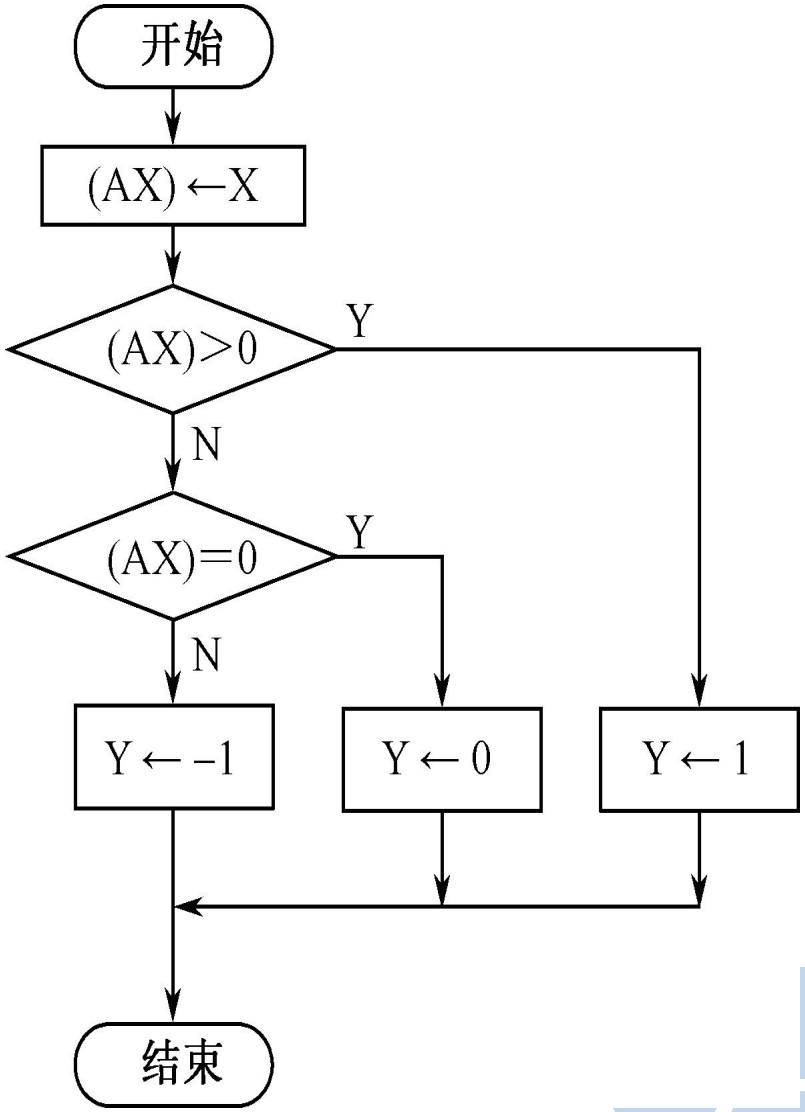
分支结构的三种形式



（a）单分支结构　　（b）双分支结构　　（c）多分支结构

- 【例5-5】　编程求分段函数Y的值。已知变量X为16位带符号数，分段函数的值要求保存到字单元Y中。函数定义如下：

$$Y=\begin{cases}1 & (\text{当}X>0)\\ 0 & (\text{当}X=0)\\ -1 & (\text{当}X<0)\end{cases}$$



```
DATA   SEGMENT     ;数据段定义
    X   DW   -128
    Y   DW   ?
DATA   ENDS
CODE   SEGMENT   ;代码段定义
    ASSUME CS:CODE,DS:DATA
START:
        MOV   AX,DATA
        MOV   DS,AX
        MOV   AX,X
        CMP   AX,0
        JG    ISPN      A>0跳转
        JZ    ISZN      A=0跳转
        MOV   Y,-1
```

```
        JMP   FINISH
ISPN:   MOV   Y,1
        JMP   FINISH
ISZN:   MOV   Y,0
FINISH:MOV    AH,4CH
        INT   21H
CODE    ENDS
        END   START
```

- 设计多分支结构程序时，应注意：
  - 要为每个分支安排出口；
  - 各分支的公共部分尽量集中，以减少程序代码；
  - 无条件转移没有范围的限制，但条件转移指令只能在-128～+127字节范围内转移；
  - 调试程序时，要对每个分支进行调试。

循环结构程序设计

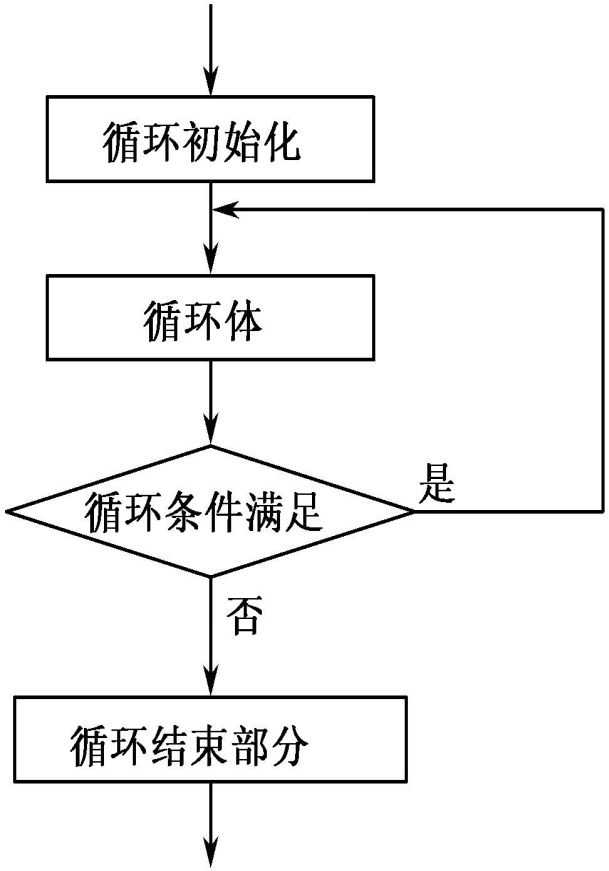
- 当需要包含多次重复执行某些相同的操作时，在程序中可使用循环结构来实现。
- 使用循环结构，可以缩短程序代码，提高编程效率。
- 循环程序由以下3个部分组成:

① 初始化部分:完成地址指针、循环计数、结束条件等初值的设置。

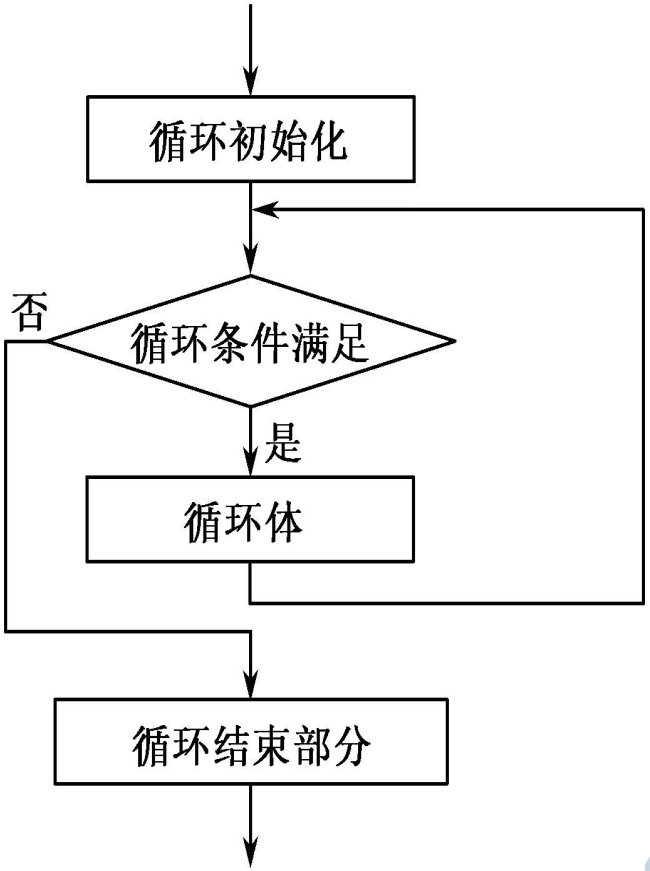
② 循环体部分：完成程序的基本操作；修改循环工作部分的变量地址等；检测和修改循环控制计数器，控制循环的运行和结束。

③ 结束部分：完成循环结束后的处理，如数据分析、结果的存放等。

循环程序结构



（a）直到型循环结构



（b）当型循环结构

- 设计循环结构程序时，要注意以下问题：

① 选用计数循环还是条件循环，采用直到型循环结构还是当型循环结构；

② 可以用循环次数、计数器、标志位、变量值等多种方式来作为循环的控制条件，进行选择时，要综合考虑循环执行的条件和循环退出的条件；

③ 注意不要把初始化部分放到循环体中，循环体中要有能改变循环条件的语句。

循环结构程序设计举例

【例5-6】 编程显示以“！”结尾的字符串。

如：“Welcome to MASM!”。

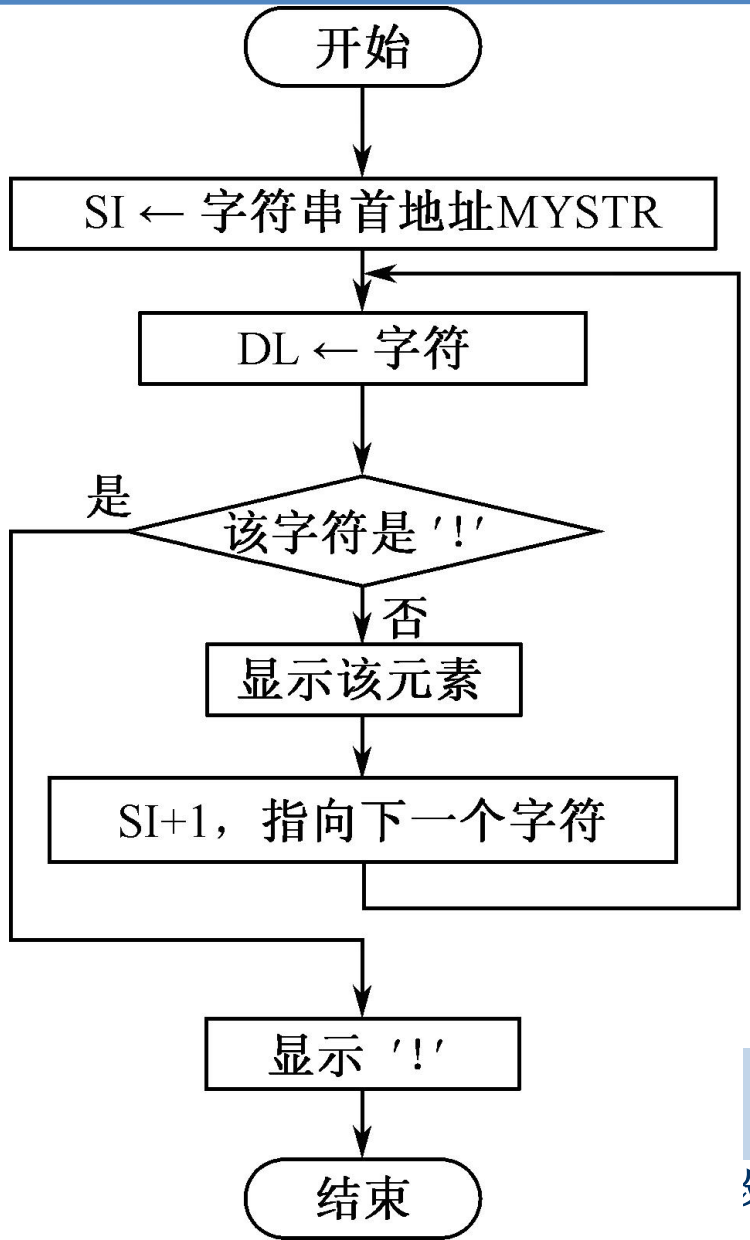
【解】

① 设计数据结构

② 设计流程

③ 由于只知道循环结束的条件是该字符串以“！”结束，不知道字符串的长度，所以，采用了条件控制的方法来控制循环的次数

④ 选择当循环结构



设计

【例5-6】（续）

```
DATA    SEGMENT
        MYSTR  DB  'Welcome to MASM!'
DATA    ENDS
CODE    SEGMENT
   ASSUME CS:CODE,DS:DATA
START:
        MOV   AX,DATA
        MOV   DS,AX
        LEA   SI,MYSTR
NEXTCHAR:
        MOV   DL,[SI]
        CMP   DL,'!'
        JZ  FINISH
        MOV   AH,2
```

【例5-6】（续）

```
        INT   21H
        INC   SI
        JMP   NEXTCHAR
FINISH:
        MOV   AH,2
        INT   21H
        MOV   AH,4CH
        INT   21H
CODE    ENDS
        END   START
```

5.4.6 子程序设计

- 子程序是可供其他程序调用的具体特定功能的程序段。
- 子程序应用可避免重复编写程序，提高代码重用率，便于规范程序的逻辑结构和分步调试等。
- 调用子程序的程序体，称为“主程序”或“调用程序”。

- 采用子程序进行程序设计的注意点：

① 现场保护和恢复

“现场保护”——子程序运行时，对可能破坏的主程序用到的寄存器、堆栈、标志位、内存数据值进行的保护。

“现场恢复”——子程序结束运行返回主程序时，对被保护的寄存器、堆栈、标志位、内存数据值的恢复。

② 子程序可嵌套

子程序中使用堆栈来保护断点，由堆栈操作的“后进先出”特性保证各个层次子程序断点的正确入栈和返回，实现嵌套。因此在子程序中对堆栈的操作要精确控制。

③ 参数传递

- 主程序在调用子程序时，经常需要向子程序传递一些参数或控制信息，子程序执行完成后，也常常需要把运行的结果返回给调用程序，这种调用程序和子程序之间的信息传递，称为“参数传递”。
- 参数传递的主要方法有：寄存器传递、内存变量传递和堆栈传递。

④ 4. 编写子程序调用方法说明, 子程序调用方法说明包括：

- 子程序功能
- 入口参数
- 出口参数
- 使用的寄存器或存储器及调用实例。

子程序设计举例

【例5-8】 利用寄存器传递参数。编写子程序，实现以二进制形式显示BX的值（假设为无符号数）。

【解】

① 设计数据结构

② 设计流程

```
;-------------------------
;子程序名:DISP_BINARY
;功能:以二进制形式显示BX的值(假设为无符号数)
;入口参数:BX
;出口参数:无
;-------------------------
DISP_BINARY   PROC
     PUSH   CX              ;保护现场
     PUSH   DX
     PUSH   AX
```

【例5-8】（续）

```
        PUSHF
        MOV   CX,16
NEXTCHAR:
        ROL   BX,1
        MOV   DL,BL
        AND   DL,1
        OR    DL,30H
        MOV   AH,2
        INT   21H
        LOOP  NEXTCHAR
```

【例5-8】（续）

```
FINISH:
        MOV   DL,'B'
        MOV   AH,2
        INT   21H
        POPF                        ;恢复现场
        POP   AX
        POP   DX
        POP   CX
        RET
DISP_BINARY  ENDP
```

- 本例利用寄存器BX传递参数，作为入口参数的寄存器可以保护也可以不保护；
- 作为出口参数的寄存器是不能保护的，否则就失去了传递参数的作用。
- 由于寄存器的数量有限，只适用于少量数据的传递。

子程序设计举例（续）

【例5-9】 利用指定存储单元进行参数传递，编程利用子程序实现数据块的复制。

```
SSEG   SEGMENT
         DW   64   DUP(?)
    TOS  LABEL   WORD
SSEG   ENDS
DATA   SEGMENT
         BUF1        DB  1,2,3,4,5,6,7,8,9,100
         BUF2        DB   10      DUP(?)
         SRCADDR     DW   ?
         DSTADDR     DW   ?
         LEN         DW   ?
DATA   ENDS
```

【例5-9】（续）

```
CODE   SEGMENT
    ASSUME CS:CODE,DS:DATA,SS:SSEG,ES:DATA
START:
        MOV   AX,DATA
        MOV   DS,AX
        MOV   ES,AX
        MOV   AX,SSEG
        MOV   SS,AX
        MOV   SP,OFFSET TOS
        LEA   AX,BUF1
        MOV   SRCADDR,AX        ; 置源数据区首地址
        LEA   AX,BUF2
        MOV   DSTADDR,AX        ; 置目的数据区首地址
        MOV   LEN,10            ; 置数据块长度
        CALL  MOVEMYDAT         ; 调用子程序MOVEMYDAT
```

【例5-9】（续）

```
        MOV   AH,4CH
        INT   21H
;-----------------------------------------
;子程序名:MOVEMYDAT
;功能:数据块复制
;入口参数:源数据区首地址存SRCADDR
;入口参数:目的数据区首地址存DSTADDR,数据块长度存LEN
;出口参数:无
;-----------------------------------------
```

【例5-9】（续）

```
MOVEMYDAT   PROC
        MOV   SI,SRCADDR
        MOV   DI,DSTADDR
        MOV   CX,LEN
        STD
        ADD   SI,CX
        DEC   SI
        ADD   DI,CX
        DEC   DI
BEGINMOV:
        REP   MOVSB
        RET
MOVEMYDAT    ENDP
CODE    ENDS
        END   START
```

本例利用指定存储单元进行参数传递。这种方法实现的子程序通用性较差。

【例5-10】 利用堆栈进行参数传递。

5.4.7 汇编语言程序设计举例

【例5-11】 编程计算 (W-(X*Y+Z-200))/25，其中X，Y，Z和W都是带符号16位数 ，计算结果的商存入AX，余数存入DX。

【解】

```
SSEG  SEGMENT  STACK    'STACK'
            DW       64  DUP(?)
      TOS   LABEL    WORD
SSEG  ENDS
DATA    SEGMENT
      X     DW    6
      Y     DW   -7
      Z     DW   -280
      W     DW   2011
DATA    ENDS
```

【例5-11】（续）

```
CODE     SEGMENT
    ASSUME CS:CODE,DS:DATA,SS:SSEG,ES:DATA
START:
    MOV   AX,DATA
    MOV   DS,AX
    MOV   ES,AX
    MOV   AX,SSEG
    MOV   SS,AX
    MOV   SP,OFFSET TOS
    MOV   AX,X
    IMUL   Y
    MOV   CX,AX
    MOV   BX,DX            ;(BX,CX)←X×Y
    MOV   AX,Z
```

【例5-11】（续）

```
        CWD                     ;(DX,AX)←把Z扩展为双字类型
        ADD   CX,AX
        ADC   BX,DX             ;(BX,CX)←X×Y+Z
        SUB   CX,200
        SBB   BX,  0            ;(BX,CX)←X×Y+Z-200
        MOV   AX,W
        CWD                     ;(DX,AX)←把W扩展为双字类型
        SUB   AX,CX
        SBB   DX,BX
        MOV   BX,25
        IDIV   BX
        MOV   AH,4CH
        INT   21H
CODE    ENDS
        END   START
```

【例5-15】 请用冒泡排序法编程将内存ARRAY单元开始存储的一组8位带符号数据按从小到大排列。

【解】算法？P105-106，绘出流程图

寄存器功能说明如下：

① CH←外循环比较（轮数）计数值；

② SI←数据区地址偏移量；

③ CL←内循环比较（次数）计数值；

④ BL←交换标志。

```
DATA    SEGMENT
            ARRAY   DB      12,87,-51,68,0,15
            LEN     EQU     $-ARRAY
DATA    ENDS
```

【例5-15】（续）

```
CODE    SEGMENT
    ASSUME      CS:CODE,DS:DATA
START:
    MOV   AX,DATA
    MOV   DS,AX
    MOV   BX,LEN-1          ;BX←比较轮数
LOP0:
    MOV   SI,LEN-1          ;SI←第N个数据在数据区的偏移地址
    MOV   CX,BX             ;CX←比较次数计数值
    MOV   DX,SI             ;DX←置交换标志为第N个数据的偏移地址
LOP1:
    MOV   AL,ARRAY[SI]
    CMP   AL,ARRAY[SI-1]          ;相邻两数据比较
    JGE   NEXT
    MOV   AH,  ARRAY[SI-1]        ;TABLE[SI]←→TABLE[SI-1]
    MOV   ARRAY [SI-1],AL
    MOV   ARRAY [SI],AH
```

【例5-15】（续）

```
    MOV   DX,SI          ;DX←发生交换处的位置,给交换标志
NEXT:
    DEC   SI             ;修改数据地址
    LOOP  LOP1           ;控制内循环比较完一轮吗?
    CMP   DX,LEN-1       ;需要下一轮吗?
    JZ    FINISH         ;不需要下一轮,已全部排好序
    DEC   BX             ;控制外循环所有轮都比较完否?
    JNZ   LOP0           ;未完继续
FINISH:
    MOV   AH,4CH
    INT   21H
CODE    ENDS
    END   START
```

5.5汇编语言程序的上机过程

1. 编辑程序：EDIT.COM或其他文本编辑工具软件，用于编辑源程序。
2. 汇编程序：MASM.EXE，用于汇编源程序，得到目标程序。
3. 连接程序：LINK.EXE，用于连接目标程序，得到可执行程序。
4. 程序运行：
5. 调试程序：DEBUG.EXE，用于调试可执行程序。

【课练】 比较两个字属性的符号数X、Y的大小。

当X>Y时，AL置1；

当X=Y时，AL置0；

当X<Y时，AL置为-1。

【解】：程序段如下

```
        MOV     BX,X
        CMP     BX,Y
        JG      SET1
        JZ      SET0
        MOV     AL,-1
        JMP     EXIT
SET1:   MOV     AL,1
        JMP     EXIT
SET0:   MOV     AL,0
EXIT::  …
```

思考题：1、2、3、4、18

习题：5、6、8、9、10、12、13、16、22